

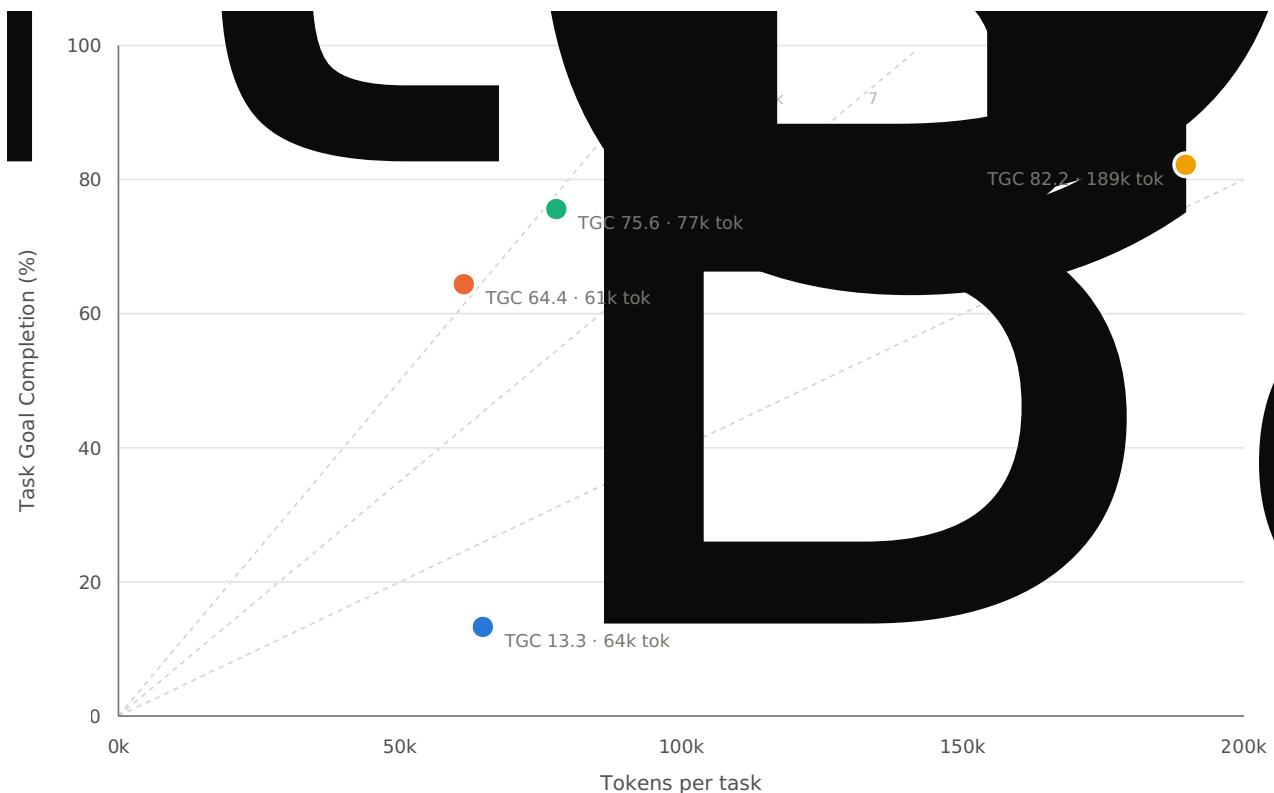
LARA Architecture Ablation: How Much Does Specialist Knowledge Buy?

Measuring LARA against the official AppWorld baseline agent, then varying how much per-app specialist knowledge the Executor sees and how it is delivered — accuracy and token cost, gpt-4.1-mini, 45-task slice

Bottom line

Against the **official AppWorld baseline agent**, LARA raises TGC from **13.3 to 75.6** (6/45 → 34/45) on the same slice, same model, same day. The improvement is a strict superset — every task the baseline solved, LARA also solved — and is the largest and most significant effect in this study (McNemar $p < 0.0001$). The baseline's failures are dominated by no_submit (23/39): without a plan supplying real API names, gpt-4.1-mini burns its entire step budget hallucinating endpoints that do not exist.

Varying the specialist knowledge produces a **monotonic gradient** — **64.4 → 75.6 → 82.2** as the Executor sees none, one app's, or all ten apps' knowledge. The knowledge matters: removing it entirely costs 17.8 points against the all-apps arm ($p = 0.02$). **How it is delivered does not:** per-step routing versus flat concatenation is indistinguishable in accuracy (75.6 vs 82.2, $p = 0.51$) while costing **2.44× fewer tokens**. The original hypothesis — that routing improves accuracy by reducing distraction — is **not supported**; what routing buys is context efficiency, not correctness.



Dashed lines are iso-efficiency contours (tasks solved per million tokens); a point above a line beats that efficiency. Dispatch and generic sit near the 10-per-million contour, monolith near 4, the baseline near 2.

What was ablated

Three arms on the same pipeline and slice:

Arm	What it is	Isolates
A — baseline	AppWorld's official minimal ReAct agent: the upstream onboarding prompt, one LLM, a plain REPL loop. No plan, no specialists, no ledger, no helpers.	the floor LARA is built on
B — dispatch (shipped LARA)	full pipeline: Explorer plan, per-step specialist routing, cross-app ledger	reference
C — monolith	the <i>routing</i> removed, not the knowledge: all 10 specialist prompts concatenated into one 76,578-char system prompt used at every step	whether per-step <i>selection</i> matters, at equal total knowledge
D — generic	the specialist knowledge removed entirely: the base ReAct prompt only, 7,025 chars, no per-app blocks. Plan and ledger retained.	whether the specialist prompts contribute anything at all

Arms C and D separate two questions that a single ablation would confound. C holds the knowledge constant and removes only the *routing*; D removes the *knowledge* while keeping routing machinery and everything else. Together they say which of the two the shipped design is actually relying on.

Arm A deliberately bypasses LARA's tooling: code goes straight to `world.execute()`, not through `tools.execute_python_code`, whose docstring carries LARA's own authentication pattern. It also skips `BOOTSTRAP_CODE` and the ledger. Routing through either would have handed the baseline one of LARA's contributions and made it a LARA variant.

Each arm's configuration was confirmed from its own run log before scoring, not assumed from config: arm B shows routing varying across steps (SpotifyExecutor 186, PhoneExecutor 46, VenmoExecutor 44, FileSystemExecutor 36, SimpleNoteExecutor 31); arm C shows `_FixedPromptSpecialist` 348 times and nothing else; both show 45 Explorer invocations, arm A none.

Method

All three arms ran the identical fixed 45-task slice (`extended_slice.json`, train only), in identical task order, on gpt-4.1-mini via the same API gateway, on the same day (2026-09-04). Paired design: every task appears in every arm.

Held constant: `MAX_ITERATIONS=6`, `MAX_EXECUTOR_RUNS=1` (no retry), `MAX_REACT_STEPS=16` (arm A: `MAX_STEPS=16`), `ENABLE_REVIEWER_RETRY=False`. Between arms B and C, the Explorer's plans and their step ordering, the ledger and `BOOTSTRAP_CODE` are also identical — only the Executor's system prompt varies.

Token counts are the provider's own usage figures recorded per call (`token_meter.py`), not estimates from prompt sizes.

Task slice composition (`extended_slice.json`)

breadth	n	apps
1-app	27	spotify (21), file_system (3), phone (2)
2-app	13	phone+venmo (5), file_system+spotify (3), simple_note+spotify (3)
3-app	5	phone+simple_note+venmo (3), file_system+phone+venmo (2)

difficulty	n
1	13
2	16
3	16

25 scenarios. Breadth is derived from which specialists arm B actually routed to. The slice is spotify-heavy (21/45 single-app spotify), which under-weights exactly the multi-app tasks where dispatch should matter most.

Results

arm	specialist knowledge	TGC	SGC	d1	d2	d3	tok/task	solved/1M tok
A — baseline	none (no plan/ledger either)	13.3 (6/45)	4.0	23.1	12.5	6.2	64,752	2.1
D — generic	none	64.4 (29/45)	56.0	69.2	68.8	56.2	61,382	10.5
B — dispatch	current app only	75.6 (34/45)	68.0	84.6	62.5	81.2	77,818	9.7
C — monolith	all 10 apps	82.2 (37/45)	80.0	92.3	75.0	81.2	189,666	4.3

Baseline vs dispatch (paired, exact McNemar): 28 tasks solved only by dispatch, **0** only by baseline, 6 by both, 11 by neither — **p < 0.0001, significant**. A strict superset.

Dispatch vs monolith: 6 tasks solved only by monolith, 3 only by dispatch, 31 by both, 5 by neither — **p = 0.5078, not significant**. Fewer than 10 discordant pairs: directional only. Difficulty-3 is an exact tie (13/16 both), which is where dispatch should help most if it helps anywhere.

Generic vs monolith: 9 tasks solved only by monolith, 1 only by generic — **p = 0.0215, significant**. This is the only specialist-knowledge comparison that reaches significance at n=45: it takes the full knowledge gap (none vs all ten apps) to produce a detectable effect. **Generic vs dispatch** (8 vs 3, p = 0.2266) points the same way without reaching it.

Generic's failures are wrong answers rather than non-submission (its no_submit count matches dispatch's), and its weakness is concentrated at difficulty-3 (56.2 vs 81.2) — the multi-app tasks where exact field names matter most. That is consistent with the blocks' stated purpose: supplying correct API surface, not driving the agent to finish.

Note the opposite difficulty gradients: the baseline collapses across levels (23.1 → 12.5 → 6.2) while LARA holds or rises (84.6 → 62.5 → 81.2). The architecture's value grows with task complexity.

Token cost

	baseline	generic	dispatch	monolith
total	2,913,834	2,762,206	3,501,820	8,534,948
per task	64,752	61,382	77,818	189,666
LLM calls / task	14.3	10.7	11.1	10.6
Executor input	—	~1.1M	2,162,667	7,331,786
Explorer input	—	~1.2M	1,267,527	1,130,547
solved / 1M tokens	2.1	10.5	9.7	4.3

Generic is the most token-efficient arm (10.5 solved per million), reaching 85% of dispatch's accuracy for 79% of its cost. The baseline is comparably cheap per task yet least efficient per result — it spends tokens failing, averaging the most LLM calls (14.3) because it exhausts its step budget. Monolith buys the top accuracy at $3.1\times$ generic's cost.

If tokens are the binding constraint, generic is the defensible configuration; if accuracy is, monolith is. Dispatch sits between them on both axes.

Between B and C the Executor input ratio is **$3.39\times$** ; overall cost separates by only $2.44\times$ because the Explorer (~1.2M tokens in both) and tool output are identical by construction — the Explorer alone is **36% of dispatch's total input**, the largest single optimisation target in the system and unrelated to specialists.

The $5.5\times$ system-prompt size ratio (76,578 vs ~13,980 chars) **overstates real cost**; conversation history and tool output dominate actual consumption. Use the measured $2.44\times$.

Mechanism

Arm A's collapse is an API-discovery failure. 23 of 39 baseline failures are `no_submit` — `complete_task()` never called; 23 tasks exhausted the 16-step budget. The logs show why: the model invents endpoints that do not exist —

```
No API named 'set_access_token' found in the simple_note app
No API named 'show_account_usernames' found in the supervisor app
No API named 'list_transactions' found in the venmo app
```

— and spends its budget discovering that they are wrong. LARA's Explorer does this discovery in separate LLM calls that never touch the Executor's step budget, and its specialists carry the correct names outright. This reproduces the failure signature reported for the plan-removal arm in the model-tier study.

Arm C's null result contradicts the predicted mechanism. The prediction was attention dilution: with all 10 apps in context, conventions from adjacent apps bleed together. The data do not support it. Monolith's `no_submit` rate is identical to dispatch's (2 tasks each), its 76,578-char prompt produced **no** context-length errors, and there was no latency penalty (25.3 vs 27.4 min wall clock). `gpt-4.1-mini` appears able to attend to the relevant block within a 76 KB prompt well enough that pre-selecting it wins nothing in accuracy. What pre-selection wins is the $3.39\times$ not spent re-sending nine irrelevant blocks at every ReAct step.

Held-out confirmation on `test_normal`

The arms above run on a 45-task train slice. The same baseline agent was also scored once on the full official `test_normal` split (168 tasks), aggregate only — no per-task reports were opened, since the AppWorld rules permit scoring the held-out splits but not using them for error analysis:

agent	model	TGC	SGC	d1	d2	d3
official baseline	gpt-4.1-mini	23.2	7.1	47.4	18.8	4.8
official baseline	claude-opus-4-7	54.8	46.4	87.7	52.1	27.0
LARA	gpt-4.1-mini	61.9	50.0	—	—	—
LARA	claude-opus-4-7	88.7	82.1	98.2	89.6	79.4

With the model held fixed the architecture is worth **2.7×** on gpt-4.1-mini (23.2 → 61.9) and **1.6×** on claude-opus-4-7 (54.8 → 88.7). The scaffold helps more where the model is weaker, but does not become redundant at the frontier tier: 33.9 TGC points and 35.7 SGC points remain.

The baseline's failure profile on test_normal matches the train slice: 91 of 129 failures (70%) are no_submit — step-budget exhaustion without calling complete_task(). Cost was 10.48M tokens over 168 tasks (62,361/task), in line with the 45-task run's 64,752.

The stale-baseline correction

An earlier version of the B-vs-C comparison used a dispatch baseline of **TGC 64.4** recorded on 2026-08-19 and concluded that monolith *outperformed* dispatch by 11 points. Re-running dispatch on 2026-09-04 with **no code changes** produced 75.6 — the entire apparent advantage was drift in the hosted gpt-4.1-mini, not architecture.

Any cross-day comparison against a hosted model is unsafe. All numbers above are same-day. This also affects the planning-separation study, whose 64.4-vs-33.3 headline uses the same August baseline; both arms drifted together so the direction likely holds, but the magnitude should not be quoted until re-run. August data is preserved in ablation_full_ext_AUG2026 and ablation_single_ext_AUG2026.

Caveats

- **Run-to-run variance exceeds the B-vs-C effect.** Two identical monolith runs on consecutive days scored 75.6 and 82.2 — 6.6 points apart, larger than the gap to dispatch. No accuracy claim between B and C survives this in either direction; n=45 is underpowered for effects of this size. The A-vs-B gap (62 points, 28 discordant pairs) is far outside this noise band and is unaffected.
- **One run per cell.** No within-day repeats, so the variance above is measured across days and confounded with drift.
- **Single model tier.** Only gpt-4.1-mini. The dispatch mechanism plausibly matters more for weaker models that cannot locate the relevant block in a long prompt, and less for stronger ones — untested here. A separate model-tier study found the architecture contributes little on claude-opus-4-7 and a great deal on gpt-4.1/gpt-4o.
- **The knowledge gradient is confounded with prompt length.** Arms D/B/C differ in both *how much* specialist knowledge is present and *how many tokens* the system prompt costs. A length-matched control — irrelevant filler padded to 76,578 chars — would separate "more knowledge" from "longer prompt"; it was not run.
- **Slice composition under-weights the B/C mechanism.** 27/45 tasks are single-app, where routing and concatenation are nearly equivalent by construction. Only 5 are 3-app.
- **Monolith is naive concatenation**, not a well-engineered single prompt. A compressed all-apps prompt would be a fairer and more demanding baseline.
- **Alphabetical block order** is fixed, so Amazon's 18,729 chars always lead and position effects are not averaged out.
- Train split only; held-out test splits were not touched.